



NK Advanced Collector Pro

A Multi-Threaded Asset Collection & Archiving Tool for Foundry Nuke

VERSION 1.0.0

Author: Nitin Kashyap - Junior VFX Compositor

Engine: Python 3 / PySide6 (PySide2 fallback) - 69 supported node classes

License: GNU GPL v3.0

Table of Contents

Table of Contents	2
1. Introduction	3
Who this manual is for	3
2. Key Features	3
3. Requirements	4
4. Installation	4
Option A - Self-contained folder (recommended)	4
Option B - Flat / traditional .nuke layout	4
Launching the tool	5
5. Quick Start Walkthrough	5
6. Interface Guide	5
6.1 Collection tab	5
6.2 Node Filter tab	5
6.3 Validation tab	6
6.4 Settings tab	6
6.5 About tab	6
7. Settings Reference	6
8. Supported Node Types	7
9. Path Tokens	8
10. How It Works	8
Output folder structure	8
11. Headless / Render-Farm Mode	9
12. Known Limitations / Reserved Settings	9
13. Troubleshooting / FAQ	10
14. License & Credits	10
Author	10

1. Introduction

NK Advanced Collector Pro scans the current Nuke script for every node that carries a source file path, validates that those sources actually exist - and that there is room to copy them - before anything is touched, then copies all footage into a clean, self-contained archive folder using a multi-threaded copy engine, with live progress, pause/resume/cancel, an HTML collection report, and full settings persistence between sessions.

It is built as an enhanced, from-scratch implementation of the "collect / archive script" workflow popularised by tools such as cragl's smartCollect, re-engineered with a dedicated pre-flight validation pass, a per-node asset filter table, token-based output paths, and a render-farm-ready headless / command-line mode.

Who this manual is for

Compositors and pipeline TDs who need to hand off, archive, or migrate a Nuke script together with every piece of footage, geometry, camera, cache, and look-up table it depends on - with a verifiable audit trail of exactly what was copied, what was missing, and how much disk space it used.

2. Key Features

Feature	Description
Pre-Flight Validation	Checks every source path, missing sequence frames, duplicate paths, non-default colorspace, and target-disk free space before a single byte is copied.
Parallel Archiving	A ThreadPoolExecutor-backed copy engine with configurable thread count and buffer size, plus live Pause / Resume / Cancel.
Node Filter Table	A sortable, searchable asset table (format, frame range, frame count, file type, size on disk) for selectively including or excluding sources before archiving.
Token-Based Target Paths	{project} {shot} {version} {artist} {date} {time} {script} tokens resolve automatically, with auto-incrementing versioned folders if the target already exists.
Automatic Relinking	Collected nodes are relinked to portable, TCL-anchored relative paths so the archive opens correctly on any machine.
HTML Collection Report	A self-contained, dark-themed collection_report.html with the full copy log, byte counts, throughput, and pre-flight results.
69 Node Classes Supported	Read, Write, Deep, Geo, Camera, Axis, Cryptomatte, CopyCat/ML, VDB, MaterialX, and the full Nuke 17 USD/Hydra/LOP node family.
Gizmo to Group Conversion	An optional pass that converts third-party Gizmos into native Groups so the archive does not depend on plugins the recipient may not have installed.
Headless / Farm Mode	The same engine runs from the command line for Deadline / Tractor / RoyalRender-style automated collection.
Themeable Dark UI	A built-in dark theme plus a drop-in .qss theme picker, completion sound, and desktop notifications.
Full Settings Persistence	Every preference is saved to data/settings.json and restored automatically on the next launch.

3. Requirements

Component	Minimum / Notes
Foundry Nuke	12.0 or newer. Built and tested against the Nuke 17 USD/Hydra node set; degrades gracefully on older versions.
Python	3.7+ (the interpreter bundled with your Nuke installation).
Qt bindings	PySide6 (Nuke 14+), automatically falls back to PySide2 (Nuke 12-13).
Operating System	Windows, macOS, or Linux.
External packages	None. Everything used ships with Nuke's interpreter.

4. Installation

Option A - Self-contained folder (recommended)

Keeps the whole tool - script, icons, sounds, styles, and settings - as one portable, drop-in folder.

- **Step 1.** Download or clone the repository.
- **Step 2.** Copy the entire `AdvancedCollector_Pro/` folder into your Nuke plugin path (e.g. directly into `~/ .nuke/`).
- **Step 3.** Open (or create) `~/ .nuke/init.py` and register the path.
- **Step 4.** Restart Nuke.

```
import nuke
nuke.pluginAddPath('./AdvancedCollector_Pro')
```

`pluginAddPath` adds the folder to `sys.path` and tells Nuke to auto-execute any `menu.py` it contains, so the menu entries register themselves automatically and `icons/`, `sounds/`, `styles/` and `data/settings.json` are all found right next to the script (the tool auto-detects this and uses the folder itself as its base directory).

Option B - Flat / traditional .nuke layout

If you would rather not edit `init.py`, install the pieces manually, exactly as described in the comments at the top of `menu.py`:

- Copy `AdvancedCollector_Pro.py` directly into `~/ .nuke/AdvancedCollector_Pro.py` (flat, not in a subfolder).
- Copy `icons/`, `sounds/`, `styles/` and `data/` into `~/ .nuke/AdvancedCollector_Pro/` (a same-named subfolder).
- Copy the contents of `menu.py` into your existing `~/ .nuke/menu.py` (or create one).
- Restart Nuke.

How the asset folder is resolved

If an `icons/` folder exists next to `AdvancedCollector_Pro.py`, that folder is used directly (Option A). Otherwise the tool falls back to `~/ .nuke/AdvancedCollector_Pro/` (Option B).

Launching the tool

- **Nuke menu bar:** NK Asset Collector > Advanced Asset Collector
- **Nodes panel / Tab menu / Node Graph right-click:** NK Asset Collector > Advanced Asset Collector
- **Keyboard shortcut:** Ctrl+Shift+C

5. Quick Start Walkthrough

- 1 Open the script you want to archive in Nuke.
- 2 Launch the tool with Ctrl+Shift+C, or via the menu.
- 3 On the **Collection** tab, choose AUTO PATH (target folder auto-detected next to your saved script, with auto-version-increment) or MANUAL PATH (type or browse, using path tokens; click **Resolve Tokens** to preview the resolved path).
- 4 Optional: on the **Node Filter** tab, click **Scan Nodes** to populate the asset table, then check/uncheck sources, or use the bulk-ignore buttons to trim the list quickly.
- 5 Click **Pre-Flight Check** to validate every path, sequence frame, and the target disk's free space. Review the results - fix anything flagged ERROR, or proceed anyway if you understand the risk.
- 6 Click **START COLLECTION**. Watch live progress, MB/s, and ETA; use Pause / Cancel as needed.
- 7 On completion: footage has been copied into <target>/footage/, collected nodes are relinked to portable relative paths, your script has been Saved As into the target folder, an HTML report is written (if enabled), and a completion dialog offers to open it.

Important - in-session changes

Collection relinks nodes (and, if enabled, converts Gizmos to Groups) on the script that is **currently open** in Nuke, then performs a Save As into the target folder. The original .nk file on disk at its original location is **not** modified - but the version you have open in the Nuke session now reflects the archived state. If you want to keep working on the original, unarchived version, reopen it from disk after collecting.

6. Interface Guide

The tool window is organised into five tabs, a live log panel, a progress bar with ETA, and the action row (Pre-Flight Check / START COLLECTION / Pause / Cancel).

6.1 Collection tab

Target directory input (Auto / Manual), folder browser, a Detect button (re-sync to the script's location) and a Resolve Tokens button, a live "Resolved:" preview, the auto-version-increment checkbox, and the Selective Frame Range group - use each node's own first/last frame, or force one custom global range across every collected sequence.

6.2 Node Filter tab

A 10-column sortable / searchable table - archive checkbox, node name, class, format, first/last frame, frame count, file type, size on disk, and filename - populated by Scan Nodes. Bulk-action buttons let you ignore free-floating

nodes, ignore disabled nodes, or enable/ignore the current selection. A separate checkbox controls whether Write node outputs are included in scanning/archiving at all (off by default).

6.3 Validation tab

Shows the results of the last Run Validation / Pre-Flight Check pass in a colour-coded table (red = ERROR, orange = WARNING, cyan = INFO), with a summary bar above it.

6.4 Settings tab

General sub-tab

Always-on-top, recreate source paths, gizmo-to-group conversion, reveal-on-finish, HTML report generation, desktop notifications, concurrent thread count, copy buffer size, source path mode, completion sound (with a folder-reveal button so you can drop in your own .wav files), UI theme picker (drop-in .qss files, same reveal button), output path mode, an "ignore nukescripts" filter field, and the relative-relink toggle.

Logging sub-tab

Verbose output, errors-only, log skipped files, per-node timing, and a log-level dropdown (INFO / WARNING / ERROR / DEBUG), plus a Clear Log button.

Click **Save and close** to persist every option to data/settings.json.

6.5 About tab

Tool name, version badge, author credit, a short description, GitHub/LinkedIn link cards, and a feature summary list.

7. Settings Reference

These are the factory defaults the tool ships with on first launch, before "Save and close" has ever been clicked. The bundled data/settings.json is a sample / last-saved configuration and will be overwritten the first time you save your own preferences. Rows marked **Reserved** are persisted but not yet wired into the collection logic - see Section 12, Known Limitations.

Key	Type	Default	Description
always_on_top	bool	true	Keeps the tool window above Nuke while it is open.
recreate_paths	bool	true	Reserved.
gizmo_to_group	bool	false	Converts custom Gizmos to native Groups after collection.
reveal_finished	bool	false	Opens the target folder in your OS file browser on completion.
gen_report	bool	true	Writes collection_report.html into the target folder on completion.
notify_desktop	bool	false	Fires an OS desktop notification on completion (macOS/Linux).
threads	int	min(cpu_count, 8)	Concurrent copy worker threads (1-99).
buf_size	string (MB)	"4"	Copy buffer size - one of 1 / 4 / 8 / 16 / 32 / 64 MB.

Key	Type	Default	Description
path_mode	string	"relative"	Reserved - relative / absolute / preserve.
sound	string	"no sound"	Completion .wav file from sounds/, or "no sound".
style	string	"Default"	Theme .qss file from styles/, or "Default" for the built-in dark theme.
target_mode	string	"next_to_script"	"next_to_script" (Auto Path) or "custom" (Manual Path).
ignore	string	"backup, bckp, annotation"	Reserved.
relink	bool	true	Relink collected nodes to relative, TCL-anchored archive paths.
log_verbose	bool	true	Reserved.
log_errors	bool	false	Reserved.
log_skipped	bool	true	Reserved.
log_timing	bool	false	Reserved.
log_level	string	"INFO"	Reserved - INFO / WARNING / ERROR / DEBUG.

8. Supported Node Types

The collection engine recognises the file-bearing knob on 69 node classes spanning Nuke's native readers/writers, 3D/geo/camera nodes, the full Nuke 17 USD/Hydra/LOP node family, and common third-party plugins - plus 24 fallback knob names so gizmos and node variants with non-standard knob names are still picked up.

Category	Node classes
Image / Deep readers	Read, DeepRead, AudioRead, Precomp
Geometry / scene readers	ReadGeo, ReadGeo2, GeoImport, UsdImport, UsdCameraImport
Cameras & Axis	Camera, Camera2, Camera3, Axis, Axis2, Axis3
Writers	Write, DeepWrite, WriteGeo
Particle / cache	ParticleCache
Colour / OCIO / LUT	OCIOFileTransform, LUTTransform
Special file knobs	Vectorfield (vfield_file), ScannedGrain (grain_file)
Live Groups	LiveGroup
Third-party / plugins	Cryptomatte, DeepRecolor, CopyCat, MLCClient, Card3D, ReadOFX, WriteOFX, BorisFX, FlameImport, ReadVDB, VDBRead, Render, DeadlineRender, proWrite, proRead, NukeStudio, TrackClip, ClipRead, GridWarp3, SplineWarp3
Nuke 17 USD / Hydra / LOP	UsdRead, UsdWrite, UsdStageRead, UsdStageWrite, UsdLayerRead, UsdLayerWrite, UsdPrimRead, UsdPrimWrite, UsdStageWithLayer, LookFileMaterialsIn, LookFileBake, HydraRender, ReadPTX, ReadParticles
Nuke 17 3D scene placement	GeoReference (file_path knob)

Category	Node classes
Nuke 17 geo export / transform	GeoExport, GeoTransform (chan_file), GeoCamera (chan_file), GeoEditCamera (chan_file), GeoCacheTransform, GeoCache
USD Material / Look	UsdMaterialBind, UsdShadeMaterial, MaterialX, ReadMaterialX

Fallback knob names

Checked when the primary knob is empty, and rewritten alongside it on every matching node: filename, proxy, vfield_file, grain_file, fbx_file, abc_file, chan_file, usd_file, lop_path, usd_path, file_path, geo_file, scene_path, layer_file, material_file, mtlx_file, look_file, cache_file, ptx_file, footage_file, input_file, output_file, lut_file, profile_file, model_file, vdb_file, alembic_file, scene_file, data_file.

9. Path Tokens

Used in both the target-directory field and the auto-detected path:

Token	Resolves to
{project}	The part of the script filename before the first underscore.
{shot}	The part of the script filename after the first underscore.
{version}	The first vNN pattern found in the script filename, or v01.
{artist}	Current OS username.
{date}	YYYYMMDD
{time}	HHMMSS
{script}	The script filename without its extension.

10. How It Works

1. SCAN Walk every node with a file-bearing knob (69 classes + 24 fallback knob names), resolve sequence vs. single-file.
2. VALIDATE Pre-flight: missing frames, duplicate paths, non-default colorspace, target disk free space -> Pre-Flight dialog.
3. COPY ThreadPoolExecutor pool copies into <target>/footage/, skip-if-identical (size + mtime) fast path, live progress / MB/s / ETA, pause & cancel supported.
4. RELINK Collected node knobs rewritten to a portable, TCL-anchored relative path: [file dirname [value root.name]]/footage/...
5. SAVE nuke.scriptSaveAs() writes the script into the target folder (original file on disk is untouched).
6. REPORT collection_report.html generated with the full copy log, throughput, and pre-flight findings; optional sound / desktop notification / auto-reveal in file browser.

Output folder structure

<target>/


```
|-- <script_name>.nk          # Saved-as copy of your script, relinked
|-- collection_report.html    # Self-contained HTML report (if enabled)
`-- footage/
    |-- <file>.ext            # Single-file sources (FBX, ABC, audio, LUTs...)
    |-- <node_name>_assets/
        |-- <file>.%04d.ext    # Frame sequences, one subfolder per node
```

11. Headless / Render-Farm Mode

The same scan, validate, copy, relink, report engine runs without a UI, for Deadline, Tractor, RoyalRender, or any other automated pipeline trigger. Pass `--target` (and any other flags) after `--` so Nuke's own argument parser ignores them.

```
# Representative example - exact Nuke executable flags vary by version/OS,
# consult your Nuke command-line / render-farm wrapper documentation.
Nuke15.1 -t AdvancedCollector_Pro.py -- --target "/archive/shot010_v01/" \
    --threads 8 --relink --report
```

Flag	Description
<code>--target <path></code>	(Required) Destination archive folder.
<code>--threads <int></code>	Concurrent copy threads (default 4).
<code>--buf <int></code>	Copy buffer size in MB (default 4).
<code>--relink</code>	Relink collected nodes to portable archive paths.
<code>--report</code>	Write <code>collection_report.html</code> into the target folder.
<code>--gizmo</code>	Convert custom Gizmos to Groups before saving.
<code>--zip</code>	Archive the entire target folder into a <code>.zip</code> after collection.
<code>--incremental</code>	Accepted for forward-compatibility - see Section 12.

12. Known Limitations / Reserved Settings

In the interest of full transparency, the following options exist in the UI and are saved to `settings.json`, but are not yet wired into the collection logic - they are reserved extension points for a future release:

- **Sources path mode** (relative / absolute / preserve) - the engine currently always relinks using the relative TCL anchor.
- **Recreate source paths** - saved, no effect yet.
- **Ignore nukescripts** filter - saved, not yet applied to scanning.
- **Logging sub-tab** (Verbose / Errors only / Skipped files / Timing / Log level) - saved, but the live log panel currently shows the same set of messages regardless of these toggles.
- **Path remapping** - both the validation and collection routines fully support (`old_prefix`, `new_prefix`) remap pairs internally, but the GUI's remap-list function always returns an empty list - there is no remap table in the UI yet.
- **CLI `--incremental`** - accepted by the headless argument parser but not yet connected to MD5 hash verification; headless mode always uses the fast size + mtime comparison. The worker engine used by the GUI does implement MD5-verified incremental copying internally - it is simply not yet exposed as a checkbox.

- **Standalone execution** - AdvancedCollector_Pro.py imports the nuke module unconditionally, so headless/standalone runs must use Nuke's own bundled Python interpreter (e.g. `Nuke -t`), not a system python3.

13. Troubleshooting / FAQ

Q: The tool doesn't appear in the Nuke menu after install.

A: Confirm `~/nuke/init.py` contains `nuke.pluginAddPath('./AdvancedCollector_Pro')` (Option A) and that the path matches exactly where you copied the folder. Verify from the Script Editor: `import AdvancedCollector_Pro; print(AdvancedCollector_Pro.__file__)`.

Q: My script changed after I ran a collection.

A: Expected behaviour - see Section 5's "Important" note. Collection relinks nodes (and optionally converts Gizmos) on the currently open session before Save As. Reopen the original file from disk if you want the pre-collection state back.

Q: Why are Write nodes not archived by default?

A: "Include Write node outputs" on the Node Filter tab is off by default to avoid accidentally re-copying potentially huge render output sequences. Enable it explicitly when you do want renders folded into the archive.

Q: Can I add my own completion sound or UI theme?

A: Yes - drop .wav files into sounds/ or .qss files into styles/. Both dropdowns rescan their folder every time you open them, no restart required. The folder-reveal buttons next to each dropdown jump straight to those folders in your OS file browser.

Q: How do I force one frame range across every sequence?

A: Collection tab, Selective Frame Range group: toggle "Force Custom Global Range" and enter Start/End to override every collected sequence's own first/last frame.

Q: Does it work with every Nuke edition?

A: The script only depends on the standard nuke Python module and the Node Graph / file-knob APIs common to every Nuke tier, so the GUI launches the same way across editions. Whether a given node class exists at all (e.g. CopyCat, MLCClient) still depends on which Nuke edition/license you are running.

14. License & Credits

NK Advanced Collector Pro is licensed under the GNU General Public License v3.0. You are free to redistribute and/or modify it under the terms of that license; see the LICENSE file included with this repository for the full text.

Author

Nitin Kashyap - Junior VFX Compositor

GitHub: <https://github.com/Nitinkashyap96>

LinkedIn: <https://www.linkedin.com/in/nitin-kashyap-8541213a6/>

NK Advanced Collector Pro v1.0.0 - User Manual